SwarmSort: A Fair Evaluation of Particle Swarm Hyperparameter Optimization for YOLOv8 Waste Detection Across Studio, Real-World, and Indian Data

Anushka Gupta, Ayush Kumar, and Vaishnavi

***Abstract*—Waste sorting needs detectors that find every item in a photograph, yet much published work classifies single objects on studio backgrounds, and metaheuristic tuning claims rarely include a random-search baseline or confidence intervals. We ask whether particle swarm optimization (PSO) trains a better lightweight waste detector than random search or library defaults at an equal budget. Six training hyperparameters of YOLOv8n were set by the defaults, the best of 32 random samples, or 8 particles over 4 rounds, each search spending 32 short proxy trainings; a check fixed in advance confirmed the proxy ranks settings like longer training (Spearman correlation 0.976, first generation). Each chosen setting was trained and tested once on held-out data, with paired-bootstrap 95% confidence intervals. The first generation used studio images; the second added real-world photographs and a test-only Indian street set. On studio data, defaults, random search and PSO tie at 0.676, 0.672 and 0.668 mean average precision at intersection-over-union 0.5 (mAP@50). With real-world training data, both searches significantly beat the defaults on the mixed test set (by 0.017 and 0.020 mAP@50) but not each other; on real-world photographs (0.298–0.310) no difference is significant, and Indian photographs score only 0.023–0.027. On the same test images, real-world training data raised real-world mAP@50 from 0.038–0.096 to 0.338–0.353 and cost 0.074–0.105 on studio images, all significantly. The optimizer matters little and the data matter most; tuning claims need equal-budget random-search baselines and confidence intervals.**

***Index Terms*—Waste detection, particle swarm optimization, hyperparameter optimization, YOLOv8, random search, real-world generalization, India.**

I. Introduction

WASTE can be recycled, composted or safely disposed of only if its materials are kept apart. India's Solid Waste Management Rules, 2016 require every waste generator to segregate waste at source into bio-degradable, non-bio-degradable and domestic hazardous streams (Rule 4), with green, white and black bins for bio-degradable, recyclable and other waste (Rule 15(h)) [1]. A camera that names each item's material could support this segregation at a smart bin, sorting line or collection point, and artificial intelligence is already an established tool in solid waste management [2], [3].

In the literature we reviewed, however, most vision work on waste sorting solves an easier problem: classifying one photograph of one object, often on a plain background, as with TrashNet [4] and its classifiers [5]. A heap, a conveyor or a street holds many items, some touching or partly hidden, and each must be located as well as named: an object-detection task. Majchrowska et al. [6], the starting point of this work, localize litter with one network and classify each detected crop with another. Multi-object waste detection is thus established, but it is a smaller strand than single-object classification, and several detection studies still rely on single-object studio images: the task and scene gap (G1).

Detectors are also sensitive to training hyperparameters such as the learning rate, momentum, weight decay and augmentation strength; the TrashNet authors blamed the 22% test accuracy of their convolutional neural network (CNN) on unsuitable hyperparameters [4]. Metaheuristics have tuned CNNs, for example particle swarm optimization (PSO) [7] for a mammography classifier [8] and a beluga whale optimizer for an InceptionV3 waste classifier [9], but the evidence is weaker than the reported gains suggest. In the studies we reviewed, metaheuristic tuning in waste vision targeted classifiers, and no waste detector had its training hyperparameters tuned by a swarm optimizer (G2). Tuned models were compared with other metaheuristics or with nothing, not with random search [10] at the same number of evaluations, and test results were single numbers without confidence intervals (G3), so a gain cannot be credited to the optimizer's guidance rather than to the search effort itself.

The data are a second weakness. Reviews find many waste-sorting vision studies set in simplified environments with artificially collected data [11] that differ from real waste [3], and accuracy depends heavily on the dataset: the same YOLOv5x, trained and tested on each dataset separately, scored 84.9 AP_{50} on PlastOPol but 63.3 on TACO [12]. A tuning gain on studio images may thus say little about street photographs. For India, where segregation at source is mandatory, the closest resource, the Dense Waste Segmentation Dataset (DWSD) [13], reports segmentation pixel accuracy only, and we found no Indian real-world waste test set with published object-detection baselines (G4).

SwarmSort tests the tuning question under strict conditions. It trains YOLOv8n [14], a nano detector small enough for a smart bin or a phone, and compares three ways (arms) of choosing six of its training hyperparameters: the library defaults (Arm A), the best of 32 random samples (Arm B) and a hand-written PSO with 8 particles over 4 rounds (Arm C).

The authors are with the Department of Computer Science and Engineering, Netaji Subhas University of Technology (NSUT), New Delhi, India.

Both searches spend 32 short proxy trainings. Each chosen setting is trained once for 100 epochs and tested once on a held-out split, with paired-bootstrap 95% confidence intervals on every difference. Generation 1 used studio-style images in six classes; Generation 2 added real-world photographs, a seventh class (OTHER) and a test-only Indian set built from DWSD. We address three research questions:

RQ1: At an equal budget of 32 proxy trainings, does PSO find YOLOv8n training hyperparameters that beat random search and the default settings on a held-out test set? (G2, G3)

RQ2: Does a short proxy training (12 epochs on 40% of the data) rank hyperparameter settings the same way longer training does? (G3)

RQ3: Does the answer to RQ1 change when the data move from studio images to real-world and Indian scenes, and how large is the studio-to-real-world drop? (G1, G4)

Our contributions are as follows.

- *A lightweight multi-object waste detector* (G1): YOLOv8n trained on multi-object studio and real-world scenes, with six classes in Generation 1 and seven in Generation 2 (Sections III-B and III-C).
- *A swarm-tuned waste detector* (G2): a hand-written PSO searches the learning-rate schedule, momentum, weight decay and two augmentation settings (Sections III-D and III-E); to our knowledge, within the literature we reviewed, no earlier waste detector had its training hyperparameters tuned by a swarm optimizer.
- *An equal-budget, three-arm protocol* (G3): a proxy-validity check with a pass threshold fixed in advance, passed with Spearman $\rho = 0.976$ ($n = 8$, Generation 1 only), a test split used once and paired-bootstrap 95% confidence intervals on every difference (Sections III-A, III-G and III-I).
- *A two-generation evaluation across domains* (G4): studio, real-world and Indian results reported separately, including detection results on DWSD, which, to our knowledge, within the literature we reviewed, had no published detection baselines. Our DWSD boxes come from mask regions that merge touching items, and the class mapping was inferred from the masks, so these results also measure a labeling mismatch. On the same test images, real-world training data raised real-world mAP@50 from 0.038–0.096 (Generation 1) to 0.338–0.353 (Generation 2), at a cost of 0.074–0.105 on studio images, all significant (Section IV-E).
- *An honest answer, with null results reported* (O5): PSO never meaningfully beat random search; its only significant edge, $+0.003$ mAP@50 on the Indian set, lies on scores near zero. Tuning gains over the defaults were absent in Generation 1, small in Generation 2 ($+0.017$ and $+0.020$ on the mixed test set) and did not reach real-world or Indian photographs. We explain these results (Section V) and release the code, search logs, result files and trained models.

Section II reviews related work and derives gaps G1–G4, Section III describes the methods, and Section IV answers RQ2, RQ1 and RQ3, in that order. Section V discusses the results, their implications and the limitations, Section VI gives a prioritized roadmap, and Section VII concludes.

II. Related Work and Research Gaps

This section reviews vision-based waste recognition, waste data from India and South Asia, and hyperparameter optimization, and then derives the research gaps.

A. Review Method

In October 2026 we searched Crossref, arXiv, Europe PMC, PubMed Central, OpenAlex and publisher pages by keyword (waste reviews, datasets and detectors; India and South Asia; metaheuristic and hyperparameter optimization) and by citation tracing from six seed papers, all discussed below [4]–[6], [8], [9], [15]†. We included vision work on waste or litter and optimization work on convolutional neural networks (CNNs), preferring peer-reviewed papers from 2016 to 2026 (classic method papers are older) and naming the status of preprints, technical reports and workshop papers kept to fill a gap. We confirmed that each reference exists (via Crossref, arXiv or publisher pages) and recorded whether we read its full text, only its abstract or only its record. Waste-vision works known to us only at the latter two levels are marked † and cited only for their existence and stated scope; method papers are cited for what they introduce. This is a targeted review, not a systematic one, so "the literature" below means the literature we reviewed.

B. Vision-Based Waste Recognition: From Classification to Detection

Much waste-sorting vision work classifies images of single objects. TrashNet, released with a Stanford course report by Yang and Thung [4], photographed one item at a time on a white posterboard (2,527 public images, six classes). In the report, a support vector machine (SVM) on SIFT features reached 63% test accuracy and a CNN trained from scratch only 22%, which the authors blamed on unsuitable hyperparameters, leaving a more thorough hyperparameter search and multi-object detection to future work. Kaya [5] truncates pretrained Xception and InceptionResNetV2 networks, reaching 89.72% test accuracy with one fixed training setting, and names TrashNet's plain backgrounds as a limitation; Sayed et al. [9] use a metaheuristic-tuned InceptionV3 (Section II-D). The smart bin of Alshammari et al. [15]† routes items with a MobileNetV3-based classifier on a Raspberry Pi, and TrashBox [16]† is presented as a classification set of 17,785 images in seven classes despite "detection" in its title. One label per image suits a single-item chute but not a heap, a conveyor or a street.

Reviews point the same way. Abdallah et al. [2]† survey 85 studies of artificial intelligence in solid waste management from 2004 to 2019. Lu and Chen [11]† find many waste-sorting vision studies preliminary, set in simplified environments and built on artificially collected data, and call for real-world complexity and open datasets. Fang et al. [3] find that most

vision studies they cover classify whole images by material, that detection studies are fewer and less accurate, and that lab pictures differ from real waste; their collected accuracies (72.8% to 99.95%) mix classification and detection, so they cannot rank methods.

Detection and segmentation datasets move the problem into cluttered scenes. TACO [17]†, a crowd-sourced preprint dataset, offered 1,500 photographs of litter in the wild with 4,784 annotations at release. ZeroWaste [18]† records deformable, translucent objects in heavy clutter on a recycling-facility sorting line, and WaRP [19]† targets hierarchical waste detection in recycling-plant images. UAVVaste [20] has 772 low-altitude drone images with 3,716 annotations; its authors find TACO's large, near-ground objects a poor fit for aerial detection. PlastOPol [12] has 2,418 crowd-sourced urban, beach and forest images with 5,300 annotations of one "litter" class. The TrashCan technical report [21] labels 7,212 deep-sea ROV images with boxes and masks, FloW [22]† covers floating waste in inland waters, partly with radar data, and MJU-Waste [23] has 2,475 RGB-D lab images of people holding waste, in one "waste" class. Aerial, underwater, conveyor, hand-held and street settings thus each have their own data, often with one generic class and mismatched class schemes.

Majchrowska et al. [6], the pipeline closest to ours and the starting point of this work, merge public datasets into a one-class "detect-waste" set and a seven-category "classify-waste" set that follows the sorting rules of Gdańsk. As direct multi-class detection performed poorly, they localize litter with EfficientDet-D2 and classify each crop with EfficientNet-B2, with hand-set hyperparameters and single-run results per source dataset on a held-out 20% described as a validation and testing set. Fulton et al. [24] compare YOLOv2, Tiny-YOLO, Faster R-CNN and SSD for underwater litter on desktop and embedded hardware. Melinte et al. [25] add boxes to TrashNet and report 97.63% precision with SSD. Liao and Juang [26] run YOLOv4-Tiny-3l at 22 frames per second on an Xavier NX for drone-based coastal litter monitoring. In a workshop paper, Bartolo et al. [27] train five detectors with privileged information on one drone dataset and test them on two others, and Sánchez-Ferrer et al. [28]† study Mask R-CNN for marine debris under data scarcity, adding synthetic scenes. Hyperparameters are a setup detail in all of these (Table I).

SwarmSort's detector belongs to the single-stage family introduced by YOLO [29], which recurs in speed-oriented waste studies [12], [20], [26], [30]; in the PlastOPol study, YOLOv5x was the most accurate of seven detectors and YOLOv5s ran at up to 5.19 frames per second on a phone. We use YOLOv8n [14] with COCO-pretrained weights [31] (Section III-C).

C. Waste Data From India and South Asia

Waste image data from India are scattered and mostly not built for detection. The earliest we found, SpotGarbage [32]† from IIT Ropar, is a smartphone app that detects and coarsely segments garbage in geo-tagged photos for citizen reporting. Kumar et al. [30] frame their YOLOv3 and YOLOv3-tiny study (6,437 self-collected images in six classes; 94.99% and 51.95% mAP on a split used for both validation and testing) around Indian waste statistics, but the images are mostly clean, with one or a few items each, and their location is not stated. AquaVision [33]†, by mainly Indian authors, detects waste in water bodies with a TACO-derived subset, and TrashBox [16]†, from NIT Karnataka, is a classification set.

The resource closest to an Indian street benchmark is DWSD [13]: 784 phone-camera images of streets, parks, lawns and garbage areas around Kolkata (Jadavpur, Garia) and Berhampore, West Bengal, with pixel masks for 14 waste classes. Its baselines report pixel accuracy only (89.39% for DeepLabv3+), with no detection results, and the paper is internally inconsistent about the dataset's name and image counts. Elsewhere in South Asia, UGV-NBWASTE [34] provides 3,600 images of eight non-biodegradable waste types from Barisal, Bangladesh, with oriented boxes but only about 1.1 objects per image and no baselines, and the pLitterStreet preprint [35]† maps street-level plastic litter from vehicle-mounted cameras in Thailand, Vietnam and Sri Lanka. Fang et al. [3] list data scarcity in developing countries among the field's limits.

D. Hyperparameter Optimization and Swarm Intelligence

Particle swarm optimization (PSO) [7] moves a population of candidate solutions (particles) through the search space; each particle's velocity combines its previous motion with pulls toward its own and the swarm's best positions. Shi and Eberhart [36] introduced the inertia weight, a factor on the previous velocity, and Clerc and Kennedy [37] derived convergence-controlling coefficients from particle trajectories, the basis of constriction-factor PSO. PSO needs no gradients and evaluates a fixed number of candidates per round, which suits costly evaluations such as network trainings; our settings are in Section III-E.

Random search [10] is the natural reference for any guided search: it samples settings independently and parallelizes freely, so an optimizer that cannot beat it at the same number of evaluations has not shown that its guidance helps. Multi-fidelity methods instead cut the cost of each evaluation; Hyperband [38] speeds up random search by giving more resources to promising configurations and stopping weak ones early. SwarmSort uses a simpler, fixed low fidelity: a short proxy training (Section III-G).

Metaheuristic tuning of CNNs appears in several domains, but the evidence we found is thin. Aguerchi et al. [8] use PSO to choose three architecture hyperparameters (filters, kernel size and stride) of a small CNN for mammogram masses, with training hyperparameters fixed; 4 particles run 3 iterations, about 12 network evaluations, with no baseline (default setting, random search or other optimizer) and a single reported result, although the authors note that a metaheuristic needs repeated runs and statistical analysis. Sayed et al. [9] give the most complete comparison we found in the waste domain: a multi-objective beluga whale optimizer (MBWO) tunes the learning rate, batch size and dropout period of InceptionV3 on TrashNet (population 30, 20 iterations, 15 epochs per evaluation); four

other metaheuristics share the population, iterations, bounds and fitness; an untuned model gives a reference (90.76% accuracy); and a Wilcoxon rank-sum test on the best search fitness over independent runs finds MBWO significantly better than three competitors but not the fourth (EO). The number of runs is not stated, random search is absent, test metrics are single numbers without intervals, and the headline accuracy differs between the abstract (97.75%) and the results table (97.57%). Mao et al. [39]† apply a genetic algorithm to a CNN that classifies recycling waste.

E. Comparative Analysis and Research Gaps

Table I compares the 15 studies read in full on the points that decide whether a tuning claim can be trusted: only two tune hyperparameters with a metaheuristic (both classifiers, one comparing tuning methods), none reports a confidence interval on a test metric, and only Bartolo et al. [27] test on public datasets unseen in training. The gaps G1–G4 follow from these patterns and Sections II-B–II-D, each stated with the evidence against it.

Gap 1 (task and scene). The classifiers of Section II-B [4], [5], [9], [15], [16]† assign one label per image, the TrashNet-based ones to one object on a plain background, and Fang et al. [3] find whole-image classification the most common approach; even some detection studies rest on near-single-object data [25], [30], [34]. The counter-evidence is substantial: multi-object waste detection is an established line of work, largely in computer-vision, robotics and sensing venues, with the datasets and pipeline of Section II-B [17]–[19], [22]†, [6], [12], [20], [21]; as our review is not systematic, we do not estimate how much smaller that strand is. The gap is: *in the literature we reviewed, waste recognition for sorting is dominated by single-label classification of one object on a plain background; multi-object detection in cluttered scenes is a smaller strand, and several detection studies still rely on single-object studio images.*

Gap 2 (optimization). Metaheuristic tuning in waste vision targets classifiers [9], [39]†, and PSO tuning of CNNs appears in other domains [8]. No waste detection or segmentation model study read in full describes a search over training hyperparameters (Table I), the only described search being over post-processing parameters [23]; in the sections we read, Fang et al. [3] mention PSO and genetic algorithms only for routing and prediction. Against this, metaheuristic CNN tuning does exist, so the gap concerns only its combination with waste detectors; two detection papers mention tuning without describing it [26], [27], so a search there cannot be ruled out; and we could not check the training setup of detection works read only as abstracts. The gap is: *in the literature we reviewed, no waste detector had its training hyperparameters tuned by a swarm optimizer; detection studies used default or hand-set values, or a tuning step they do not describe.*

Gap 3 (evidence). Where tuning is evaluated, baselines are weak: none in Aguerchi et al. [8] and only other metaheuristics in Sayed et al. [9], with single-number test metrics (Section II-D). No detection or segmentation paper read in full reports run-to-run variation, confidence intervals or significance tests on its accuracy (Table I); Bartolo et al. [27] state that statistical analysis was not applicable. Against this, Sayed et al. compare optimizers at equal population and iterations, with an untuned reference and a significance test on search fitness, so we do not claim that no study compares optimizers or tests significance. The gap is: *metaheuristic tuning in this area is compared against other metaheuristics or against nothing; we found no comparison with random search [10] at an equal evaluation budget, and test results are reported as single numbers without confidence intervals.* SwarmSort narrows this gap without closing it: with one training seed per arm and one search seed per method, its intervals cover test-set sampling only (Section V-D).

Gap 4 (generalization). The TrashNet-based classifiers never leave the studio, two reviews point to simplified, lab-like data [11]†, [3], and accuracy depends strongly on the dataset (the PlastOPol example of Section I [12]). Most detection studies test within one dataset or report new-site degradation only qualitatively (Table I), and the Indian work of Section II-C offers no detection benchmark. Against this, cross-domain tests do exist: Bartolo et al. [27], in a recent workshop paper, test on datasets unseen in training, Fulton et al. [24] on footage shifted in time and place, and Sánchez-Ferrer et al. [28]† add synthetic scenes to scarce real data; South Asian detection data exist in UGV-NBWASTE [34], without baselines, and in the pLitterStreet preprint [35]†. The gap is: *studio-to-real-world and cross-dataset evaluation is rare and recent, and we found no Indian real-world waste test set with published object-detection baselines.* SwarmSort reports studio, real-world and DWSD results separately and compares studio-only with mixed training on the same test images (Section IV-E); its DWSD boxes come from mask regions that merge touching items, with a class mapping inferred from the masks, so the DWSD results also measure a labeling mismatch.

Observation O5 (reporting of positive results). The applied papers we read present their methods as improvements; Sayed et al. [9], for example, call their optimizer the best despite its non-significant edge over EO. Partial negative findings do appear: a CNN far below an SVM [4], direct multi-class detection set aside as poor [6], synthetic data that relieve data scarcity only to a limited extent [28]†, and weaker performance on shifted data [24], [26]. We found no paper reporting that a tuning method failed to beat a simple baseline; as evidence of absence from a targeted sample, this is an observation, not a gap. SwarmSort reports and analyzes its null and negative results (Sections IV and V); Table II maps each gap to how SwarmSort addresses it.

III. Materials and Methods

Each design choice answers a gap of Section II-E: a multi-object detector instead of a classifier (G1), a swarm optimizer applied to a detector's training hyperparameters (G2), an equal-budget comparison with random search and the defaults, judged with confidence intervals on a test split used once (G3), and separate studio, real-world and Indian test sets (G4).

TABLE I
COMPARISON OF STUDIES READ IN FULL TEXT WITH SWARMSORT

Study	Task	Scenes	Hyperparameters set by	Tuning baselines	Runs / CIs	Cross-domain test	India data
Yang & Thung [4]	Cls	Studio	Manual	—	1 / no	No	No
Kaya [5]	Cls	TrashNet	Fixed	—	1 / no	No	No
Sayed et al. [9]	Cls	TrashNet	MBWO	4 metaheur. (equal), untuned; no RS	n.r. + Wilcoxon (fitness) / no	No	No
Aguerchi et al. [8]	Cls	Clinical	PSO (architecture)	None	1 / no	No	—
Majchrowska et al. [6]	Det + Cls	Real-world	Manual	None	1 / no	No	No
Hong et al. [21]	Det / Seg	Underwater	Fixed	None	1 / no	No	No
Kraft et al. [20]	Det	Aerial	Not described	None	1 / no	No	No
Córdova et al. [12]	Det	Real-world	Published defaults	None	No SDs / no	Per dataset	No
Wang et al. [23]	Seg	Lab + TACO	Fixed; CRF grid	None	1 / no	No	No
Fulton et al. [24]	Det	Underwater	Community	None	1 / no	Shifted footage	No
Melinte et al. [25]	Det	TrashNet	Manual + LR test	None	1 / no	Lab-made set	No
Kumar et al. [30]	Det	Mostly clean	Fixed	None	1 / test = val	No	Unstated
Liao & Juang [26]	Det	Aerial	"Tuned" (n.r.)	None	No	Qualitative	No
Bartolo et al. [27]	Det	Aerial	Manual	None	1 / no	Yes	No
Ali et al. [13]	Seg	Real-world	n.r.	—	1 / no	No	Yes
SwarmSort (ours)	**Det**	**Studio + real-world**	**PSO vs RS vs defaults**	**Equal: 32 proxy trainings each**	**1 seed / bootstrap 95% CIs**	**Yes (3 domains)**	**DWSD (test)**

Datasets: TrashNet [4] (single-object studio images), TrashCan [21], UAVVaste [20], PlastOPol [12], MJU-Waste [23] and DWSD [13] (a dataset paper). Cls = classification; Det = detection (one class in [6], [12]); Seg = segmentation; MBWO = multi-objective beluga whale optimizer; metaheur. (equal) = other metaheuristics with the same population size and iterations; RS = random search; CRF grid = grid search for the conditional random field only; LR test = learning-rate range test; SD = standard deviation; CI = confidence interval; n.r. = not reported; — = not applicable. "1 / no" = one reported run per configuration and no confidence intervals. "Fixed" = one setting used without a described selection procedure. "test = val" = the split used for validation also served as the test split. Per dataset = two datasets, each trained and tested separately; Qualitative = new sites described only qualitatively; Unstated = Indian authors, image location not stated. SwarmSort: 6 classes in Generation 1 and 7 in Generation 2; one training seed per arm; paired-bootstrap CIs; studio, real-world and Indian test sets. Full-text works not listed are the review of Fang et al. [3] and the UGV-NBWASTE dataset [34], which reports no model; works read only as abstracts or records are discussed in the text.

TABLE II
RESEARCH GAPS AND HOW SWARMSORT ADDRESSES THEM

Gap	How SwarmSort addresses it	Where
G1: task and scene	Multi-object detection with YOLOv8n (6 or 7 classes) on studio and real-world scenes	Sec. III-B, III-C
G2: optimization	A hand-written PSO tunes 6 training hyperparameters of a waste detector	Sec. III-D, III-E
G3: evidence	Defaults, random search and PSO at 32 proxy trainings each; proxy check with a threshold fixed in advance (Generation 1 only); test split used once; paired-bootstrap 95% CIs	Sec. III-A, III-G, III-I, IV-C, IV-D
G4: generalization	Real-world training data in Generation 2; studio, real-world and India (DWSD, test only, boxes from masks) results reported separately; both generations on the same test images	Sec. III-B, IV-E
O5: reporting	Null and negative results reported and explained, with limitations	Sec. IV, V

A. Experimental Design

Three arms choose the training hyperparameters of one detector (Fig. 1). Arm A (defaults) uses the Ultralytics defaults without search and is the reference any tuning must beat; Arm B (random search) takes the best of 32 random samples, and Arm C (PSO) the best setting found by a particle swarm of 8 particles over 4 rounds. Both search arms score candidates with the same proxy training (Section III-G), so the budget is counted in proxy trainings, not in wall-clock time, which varied between methods and cloud sessions (Section IV-F). Each arm's chosen setting is trained once with an identical recipe (Section III-H) and scored once on a held-out test split (Section III-I).

The experiment was run twice. Generation 1 uses one studio-style dataset with six classes at 416 px; Generation 2 adds real-world training photographs, a seventh class (OTHER), 640 px images and a test-only Indian set. Search space, PSO settings, seeds, recipes and evaluation code are shared, and both searches were rerun from scratch. Since classes, image size and training data differ, the two generations' scores are not one benchmark; their dedicated comparison is described in Section III-I.

B. Datasets

Table III lists every source with its licence and split sizes.

Generation 1 data. The Kaggle dataset "Garbage Detection – 6 Waste Categories" [40], from a Roboflow project, holds studio-style images with YOLO boxes in six classes: BIODEGRADABLE, CARDBOARD, GLASS, METAL, PAPER and PLASTIC. In training, 36.45% of the boxes cover less than 1% of the image, and BIODEGRADABLE holds 61.92% of the boxes against 5.61% for PAPER. The provided folders are split by class (the validation folder has 13,632 BIODEGRADABLE but 33 PAPER boxes, the test folder no GLASS box), so we pooled all images and re-split them 70/20/10, stratified by source class (seed 42), with at least 30 boxes per class in validation and test. Near-duplicates stay in one split: images count as copies if their files are identical (MD5), share a Roboflow source image, or have 256-bit difference hashes within 10 bits, or within 40 bits with a Pearson correlation of at least 0.90 between 64×64 grayscale

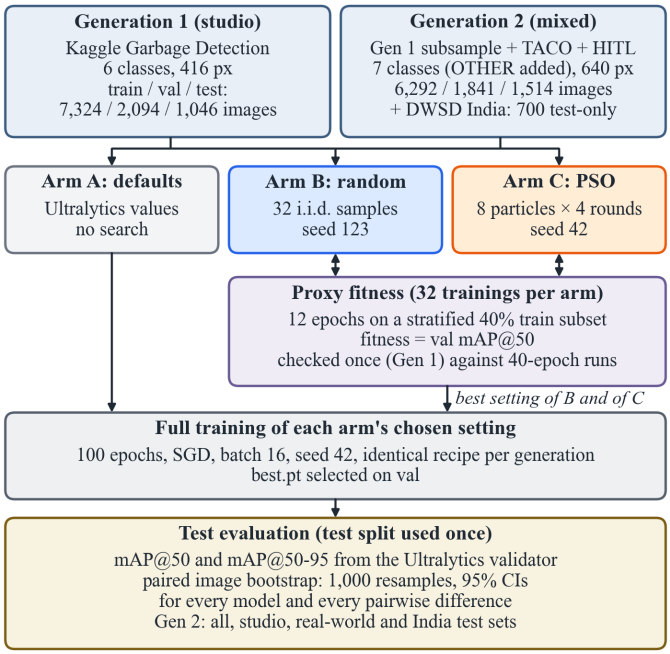


Fig. 1. Experimental pipeline shared by both generations. Arms B and C each spend 32 proxy trainings; Arm A uses the library defaults without search. Each arm's chosen setting is trained once for 100 epochs and tested once.

thumbnails. This found 47 groups (133 images). The label checks found 17 zero-size boxes and no corrupt or unlabeled image.

Generation 2 training data. Generation 2 has seven classes: the six Generation 1 classes, with unchanged indices, plus OTHER. A subsample of the Generation 1 splits, stratified by dominant class (seed 42), keeps 3,000 training and 900 validation images, and all test images are kept, so the Generation 2 studio test set is the Generation 1 test set. TACO [17] adds crowd-sourced photographs of litter in the wild; all 60 categories were mapped (for example, drink cans to METAL, cigarettes to OTHER), giving 1,260 OTHER but only 8 BIODEGRADABLE boxes. The HITL Recycling dataset [41] adds street photographs, almost all with one litter box, classed by the object tag where decisive (food waste → BIODEGRADABLE), otherwise by the material tag (aluminum → METAL), else OTHER; 100 objects lacked a material tag. TACO and HITL were split 70/20/10 with the same duplicate rules and stored with a long side of at most 640 px. The merged data have 962 duplicate groups, none spanning two sources or splits; 918 byte-identical pairs inside the real-world sources cause no leakage but give some photographs extra weight. BIODEGRADABLE holds 51.96% of the training boxes and OTHER 3.89%; the studio images have no OTHER box, and the real-world test set a single BIODEGRADABLE box.

Indian test set. The Dense Waste Segmentation Dataset (DWSD) [13] has 784 phone photographs of streets, parks and garbage areas in Kolkata and Berhampore, West Bengal, annotated as 448×448 semantic masks. We used it only for testing, never for training, search or model selection. Each 8-connected region of one mask value (of at least 50 pixels) became a box, so touching items of one class merge into one large box. The class mapping had to be inferred: the published table of mask values does not match what the regions show (the value listed as background covers polythene bags in 610 masks), so we named each value by inspecting its regions; six of the fifteen remain uncertain. The 84 images without boxes were excluded, as some show unannotated waste, leaving 3,040 PLASTIC, 1,230 PAPER, 124 CARDBOARD, 90 OTHER and 18 METAL boxes. An MD5 cross-check against the 9,647 main-build images found no identical file.

C. Detector

All arms train YOLOv8n, the nano variant of the YOLOv8 single-stage detectors [14], [29], with Ultralytics 8.4.169, starting from COCO-pretrained weights [31] with the class outputs resized to our six or seven classes. The nano model suits a smart bin or phone with limited compute and keeps 32 proxy trainings per arm affordable on one cloud GPU.

D. Search Space

Both search arms explore the same six dimensions (Table IV): the initial learning rate `lr0`, the final learning rate as a fraction of it (`lrf`), the SGD `momentum`, the `weight_decay`, the probability of mosaic augmentation (`mosaic`) and the gain of random rescaling (`scale`). The optimizers work in "search units": `lr0` and `weight_decay`, which span two orders of magnitude, are held as base-10 exponents (a particle at -3.2 means `lr0` $= 10^{-3.2}$), the others as values; decoded values are rounded to six significant digits. Three defaults lie on the boundary, `lrf` on the lower bound and `lr0` and `mosaic` on the upper bounds, so the searches reach them only at the edge. All other training hyperparameters kept the library defaults in every arm.

E. Particle Swarm Optimization

Arm C uses a global-best particle swarm optimizer [7] with an inertia weight [36], hand-written in NumPy, that maximizes the proxy fitness. Particle i has a position $\mathbf{x}_i \in \mathbb{R}^6$ in search units, a velocity $\mathbf{v}_i$ and its best visited position $\mathbf{p}_i$; $\mathbf{g}$ is the swarm's best. One update step is

$$\mathbf{v}_i \leftarrow w_t\,\mathbf{v}_i + c_1\,\mathbf{r}_1 \odot (\mathbf{p}_i - \mathbf{x}_i) + c_2\,\mathbf{r}_2 \odot (\mathbf{g} - \mathbf{x}_i), \qquad (1)$$

where $\odot$ is the element-wise product and $\mathbf{r}_1$, $\mathbf{r}_2$ are redrawn at every step (Table IV). Each component $v_{i,d}$ is then clamped to $[-v_{\max,d},\, v_{\max,d}]$ with $v_{\max,d} = 0.2\,(u_d - l_d)$, where l_d and u_d are the bounds of dimension d, and

$$\mathbf{x}_i \leftarrow \min\big(\max(\mathbf{x}_i + \mathbf{v}_i,\ \mathbf{l}),\ \mathbf{u}\big), \qquad (2)$$

element-wise; a clipped dimension's velocity is set to zero, so the particle rests at the wall. Round 1 evaluates the random initial swarm, and each of the $T - 1 = 3$ update steps is followed by one more round, so $T = 4$ rounds use exactly 32 evaluations. The inertia weight falls linearly,

$$w_t = w_{\text{start}} - (w_{\text{start}} - w_{\text{end}})\,\frac{t-1}{T-2}, \qquad t = 1, \ldots, T-1, \qquad (3)$$

TABLE III
DATASETS. COUNTS ARE IMAGES, WITH BOXES IN PARENTHESES. "CLASSES" IS THE NUMBER OF TARGET CLASSES THAT OCCUR IN THE SOURCE. THE GENERATION 2 STUDIO TEST SET EQUALS THE GENERATION 1 TEST SET; THE GENERATION 2 REAL-WORLD TEST SET IS THE UNION OF THE TACO AND HITL TEST IMAGES (468 IMAGES, 779 BOXES).

Generation	Source	Domain	Licence	Stored size	Classes	Train	Validation	Test
Generation 1	Garbage Detection (Kaggle)	studio	CC BY 4.0	416×416	6	7,324 (51,756)	2,094 (14,904)	1,046 (7,413)
Generation 2	Generation 1 subsample	studio	CC BY 4.0	416×416	6	3,000 (22,206)	900 (6,904)	1,046 (7,413)
	TACO [17]	real-world	CC BY 4.0	long side ≤ 640	7	1,050 (3,293)	301 (1,031)	149 (460)
	HITL Recycling	real-world	CC0	long side ≤ 640	7	2,242 (2,246)	640 (641)	319 (319)
	Total (7 classes)	mixed			7	6,292 (27,745)	1,841 (8,576)	1,514 (8,192)
	DWSD [13]	India	CC BY 4.0	448×448	5	–	–	700 (4,502)

TABLE IV
SEARCH SPACE AND SEARCH SETTINGS (SHARED BY BOTH GENERATIONS)

Hyperparameter	Range	Scale	Default (Arm A)
`lr0`	10^{-4} to 10^{-2}	$\log_{10}$	0.01
`lrf`	0.01 to 0.2	linear	0.01
`momentum`	0.60 to 0.98	linear	0.937
`weight_decay`	10^{-5} to 10^{-3}	$\log_{10}$	0.0005
`mosaic`	0.0 to 1.0	linear	1.0
`scale`	0.2 to 0.8	linear	0.5
Setting	**Value**		
PSO budget	8 particles $\times$ 4 rounds $=$ 32 evaluations		
Inertia w_t	0.9, 0.65, 0.4 (linear over 3 update steps)		
c_1, c_2	1.5, 1.5		
$\mathbf{r}_1$, $\mathbf{r}_2$	$U(0,1)$, fresh per particle and dimension		
Velocity limit	$\pm 20\%$ of each range		
Initialization	positions uniform in bounds; velocities uniform in $\pm 10\%$ of each range		
Boundaries	position clipped; velocity of a clipped dimension set to 0		
PSO seed	42		
Random search	32 i.i.d. uniform samples in search units; seed 123		
Fitness	validation mAP@50 after a 12-epoch proxy training on a stratified 40% training subset		

from $w_{\text{start}} = 0.9$ to $w_{\text{end}} = 0.4$. The inertia weight is the concept of [36]; this linear schedule is our setting, chosen so that the swarm explores first and refines later. A personal best is replaced only by a strictly higher fitness. All randomness comes from one seeded generator, so an interrupted search resumes exactly by replaying the seed and the logged fitness values. A failed training (non-finite mAP@50) would get fitness 0 and never be selected; none occurred in the four searches.

As a code check, with 20 particles, 50 rounds and 10 seeds on $[-5.12, 5.12]^6$, the median best value was 1.9×10^{-5} on Sphere and 6.96 on the multimodal Rastrigin function (both with minimum 0): the optimizer converges on a smooth bowl and keeps improving among many local minima. This makes no claim about tuning performance.

F. Random Search and Defaults

Arm B draws 32 points uniformly in search units (log-uniform for `lr0` and `weight_decay`) and scores them with the same proxy and script as the swarm. Random search [10] is included because any structured optimizer should beat it at the same budget. Each search arm's winner is its highest proxy fitness (the first on a tie). Arm A uses the defaults in Table IV.

G. Proxy Fitness and Its Validation

A full training takes over an hour on our GPU, so the searches score candidates with a 12-epoch proxy training on a stratified 40% training subset: 2,930 of the 7,324 Generation 1 images (by source class) and 40% of the 6,292 Generation 2 images (by source and dominant class). The subset is drawn once (seed 42) and stored as an image list, so every candidate trains on the same images. The fitness is the validation mAP@50 of the last-epoch weights. The proxy follows the full-training recipe (Section III-H), except that the no-mosaic period and the warm-up keep their share of the run, 10% and 3% (`close_mosaic` 1 epoch, warm-up 0.36 epochs), which keeps the searched `mosaic` faithful to the full run.

Before the Generation 1 searches, we tested whether the proxy ranks settings like a longer training. Seven configurations drawn by Latin hypercube sampling [42] in search units (SciPy, "random-cd", seed 42) plus the defaults were each trained with the proxy and with a 40-epoch "medium" run on the full training split (`close_mosaic` 4, warm-up 1.2 epochs), both at 416 px. The statistic was the Spearman rank correlation [43] of the eight score pairs; the pass threshold, $\rho \geq 0.6$, and the fallback, a 20-epoch proxy, were fixed in advance (results in Section IV-A).

The check was not repeated for Generation 2; the decision, recorded before its searches, relied on the Generation 1 result and the unchanged recipe, so proxy validity under the new data, classes and image size is assumed, not tested. The check also has only eight configurations and a 40-epoch rather than 100-epoch reference, and the proxy scores last-epoch weights while the final models use `best.pt` (Section V-D).

H. Full Training

Each arm's chosen setting was trained once from the COCO-pretrained weights for 100 epochs on the full training split, with an identical recipe for the three arms: SGD set explicitly (the default `optimizer=auto` would ignore the searched `lr0` and `momentum`; each run records the optimizer and learning rate used), batch size 16, a 3-epoch warm-up, no mosaic in the last 10 epochs, and 416 px in Generation 1 or 640 px in Generation 2. The checkpoint with the best validation fitness (`best.pt`) was tested; other arguments kept their defaults.

Every run used seed 42 in deterministic mode. A module that Ultralytics imports lazily during a process's first training draws random numbers after seeding, so first runs differed from later ones; importing it before seeding removed the difference. All trainings ran on one NVIDIA Tesla T4 GPU on Kaggle, with one training seed per arm, so run-to-run variance is not measured (Section V-D).

Two Generation 2 runs froze, most likely because the session's data mount stopped responding. Arm C was resumed from its epoch-55 checkpoint after checking every saved training argument against the plan; this continues the same weights, optimizer state and epoch counter but is not bit-identical to an uninterrupted run. The random search, stopped after 11 of 32 evaluations, was continued by replaying its log, which affected no completed evaluation.

I. Evaluation Protocol

Each test split was scored once, after all training; trial evaluations used the validation split. In Generation 2, one run scored the three models on the whole test split ("all", 1,514 images) and separately on the studio, real-world and India sets. Each checkpoint first had to match its arm's record (classes, epochs, image size, the six hyperparameters).

The Ultralytics validator ran at each model's training image size (batch 32, rectangular batching, confidence threshold 0.001, NMS IoU 0.7, 32-bit floats). We report mAP@50, mAP@50–95 (IoU 0.50 to 0.95 in steps of 0.05), precision and recall at the maximum-F1 confidence, and per-class AP@50. Both mAP values, recomputed from the stored per-image statistics, had to match the validator within 10^{-4}; they agreed exactly everywhere.

Uncertainty comes from a paired image bootstrap [44]: 1,000 resamples of the test images (seed 0), with the same indices for every model. We report 95% percentile intervals for each model and each pairwise difference; a difference is significant only if its interval excludes 0. Each Generation 2 test set has its own bootstrap. No p-values or multiple-comparison corrections were computed, per-class AP has no interval, and the intervals cover test sampling only, not training or search randomness.

To compare the generations on the same images, all six final models (three arms per generation) were scored on the studio and real-world test sets, each at its own training image size, with labels restricted to the six shared classes: OTHER labels were removed (images left empty stay in), and Generation 2's OTHER predictions were dropped in non-maximum suppression. Each test set has its own bootstrap. The results are in Section IV-E and Table VIII.

Inference speed is the median batch-1 latency over 100 test images after 10 warm-up predictions (confidence 0.25) at the training image size, on a laptop CPU (Intel Core Ultra 5 125H) for Generation 1 and on the T4 GPU for Generation 2, so it is comparable only within a generation.

IV. Results

We report the validity of the proxy (RQ2), the behavior of the searches and the test comparison of the three arms (RQ1), and the change from studio to real-world and Indian photographs (RQ3); interpretation follows in Section V.

TABLE V
Proxy-Validity Check (Generation 1, Validation mAP@50)

Setting	Proxy	Rank	Medium	Rank
lhs1	0.08687	1	0.28423	1
lhs2	0.23077	2	0.49451	2
lhs3	0.29495	3	0.56712	4
lhs5	0.30186	4	0.55195	3
lhs7	0.37560	5	0.61246	5
lhs6	0.44285	6	0.62161	6
Defaults	0.47648	7	0.62884	7
lhs4	0.49547	8	0.63839	8

Spearman $\rho = 0.976$, $n = 8$; pass threshold $\rho \geq 0.6$

Proxy: 12 epochs on the stratified 40% training subset. Medium: 40 epochs on the full training split. Both at 416 px; rank 1 is the lowest score. lhs1–lhs7 are the Latin hypercube settings.

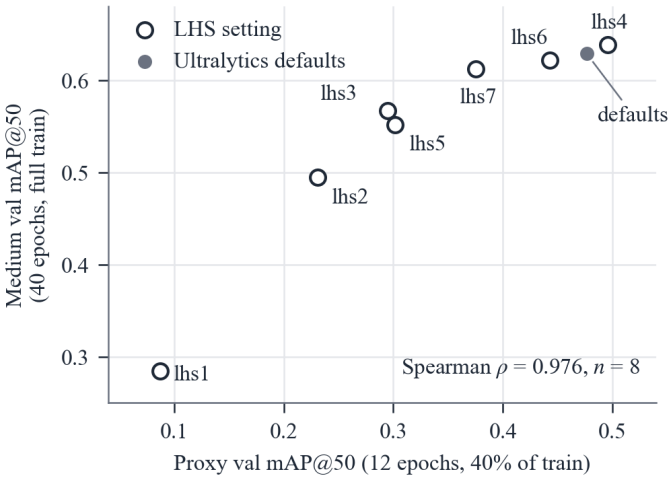


Fig. 2. Proxy versus medium validation mAP@50 for the eight settings of the Generation 1 proxy-validity check. The rankings agree except for the adjacent pair lhs3 and lhs5.

A. Proxy Validity

The proxy passed the check fixed before the Generation 1 searches (Section III-G): the Spearman rank correlation between the proxy and the 40-epoch medium scores is $\rho = 0.976$ ($n = 8$), well above the pass threshold of 0.6 (Table V, Fig. 2). The two rankings differ only by a swap of the adjacent lhs3 and lhs5. The same four settings lead both rankings, with the defaults second behind lhs4 on both, but their medium scores lie within 0.026 of each other, so the proxy separates poor settings from good ones far more clearly than good ones from each other.

A second, indirect check uses configurations that both generations evaluated. With the same seeds and search space, random search drew the same 32 configurations in both generations. PSO's positions depend on the fitness only through the personal and global bests, which coincided long enough for the swarm to visit identical positions in rounds 1–3 (24 evaluations) but not in round 4. On these repeated configurations, the proxy scores of the two generations have Spearman correlations of 0.991 (random search) and 0.892 (PSO),

recomputed from the four search logs. This compares proxy with proxy, not proxy with longer training: the generations order settings alike, but this does not replace a proxy-validity check for Generation 2.

B. Search Behavior

Fig. 3 shows every proxy evaluation and the best score so far. In Generation 1, random search found its best setting (0.4995 proxy mAP@50) at its second evaluation and never improved on it. PSO improved in steps, from 0.4894 after round 1 to 0.4942 after round 3 and 0.4953 in round 4 (evaluation 27), and the swarm converged as intended: the round means rose from 0.393 to 0.423, 0.450 and 0.482, and the within-round standard deviation fell from 0.122 to 0.088, 0.047 and 0.011. The defaults scored 0.4765 on the same proxy, above 28 of the 32 random-search and 21 of the 32 PSO evaluations. The two winners differ by only 0.0042, and lhs4 of the proxy check (0.4955) lies between them. All four search winners of the two generations use a smaller initial learning rate than the default of 0.01 (0.0025 to 0.0070).

In Generation 2, random search again peaked early (0.3340 at evaluation 3). PSO's best, 0.3290, was the last particle of round 1 and was never beaten (round bests 0.3176, 0.3237 and 0.3279 afterwards), although the swarm still converged: the round means rose from 0.267 to 0.286, 0.300 and 0.319, and the within-round standard deviation fell from 0.082 to 0.006. In both generations, PSO's mean proxy score over all 32 evaluations exceeded that of random search (Generation 1 0.437 against 0.333; Generation 2 0.293 against 0.226), which reflects the swarm concentrating its later evaluations near good points rather than finding better ones. Generation 2 has no proxy run of the defaults, so their standing on the Generation 2 proxy is unknown.

C. Generation 1 Test Results (Studio Data)

Table VI gives the one-time test results of the three Generation 1 models on the 1,046 studio test images (7,415 boxes as counted by the evaluator; Table III lists 7,413 because it leaves out 2 zero-size boxes). The arms score 0.676 (A), 0.672 (B) and 0.668 (C) mAP@50, and every pairwise 95% interval contains 0 on both metrics; the largest difference is C−A, -0.008 [-0.021, $+0.007$]. On studio data, the three arms are therefore statistically tied, and the point estimates even place the defaults first and PSO last; precision and recall are nearly identical. Validation points the same way: the defaults and PSO both reached 0.657, while random search, whose setting had the highest Generation 1 proxy score, reached the lowest value, 0.643.

Per class (Fig. 4; per-class AP has no confidence interval, Section III-I), the arms differ by less than 0.02 AP@50 on five of the six classes. The exception is PLASTIC, where PSO scores 0.638 against 0.699 (A) and 0.693 (B); PSO is highest on BIODEGRADABLE, GLASS and METAL. The three networks share one architecture, so the higher CPU latency of B (Table VI) most likely reflects other load on the laptop during timing.

D. Generation 2 Test Results (Mixed Data)

Table VII reports the one-time Generation 2 test results on the whole test split and on each domain. On the whole split, the defaults score 0.479, random search 0.497 and PSO 0.500 mAP@50. Both tuned arms are significantly better than the defaults (B−A $+0.017$ [$+0.006$, $+0.030$], C−A $+0.020$ [$+0.011$, $+0.032$]), whereas PSO and random search cannot be told apart (C−B $+0.003$ [-0.008, $+0.015$]). The single-arm intervals overlap widely; the gains are resolved only because the paired bootstrap cancels the variation in image difficulty that all three models share. The mAP@50–95 verdicts agree with the mAP@50 verdicts everywhere. Random search led PSO on validation (0.473 against 0.466), but on test the two tie.

Studio images contribute 7,413 of the 8,192 boxes, and the studio test set alone (the 1,046 Generation 1 test images) gives the same picture: B−A ($+0.024$) and C−A ($+0.019$) are significant, C−B (-0.005) is not. PSO and random search thus differ significantly only on the Indian test set, where all scores are near zero (Section IV-E).

Per class on the whole split, PSO has the highest AP@50 on five of the seven classes, random search on BIODEGRADABLE and PLASTIC, and the defaults on none. OTHER is very weak for every arm (AP@50 0.050–0.063); all 132 OTHER test boxes are in the real-world images. Precision rises from 0.549 (A) to 0.563 (B) and 0.582 (C), and recall from 0.445 (A) to 0.470 for both tuned arms. The median latency on the T4 at 640 px is 8.1 to 8.7 ms per image.

E. Domain Gap

Averaged over the three Generation 2 arms, test mAP@50 falls from about 0.59 on studio images to 0.31 on real-world and 0.03 on Indian images (Fig. 5); for every arm, the real-world score is roughly half the studio score.

On the 468 real-world test images (TACO and HITL), no pairwise difference is significant (Table VII). The intervals are wide and lopsided because BIODEGRADABLE has a single box in this set: whether a resample leaves out or repeats that image swings that class's AP, one seventh of the mean, and the arms score 0.249 (A), 0.020 (B) and 0.003 (C) AP@50 on that box. With B−A and C−A intervals 0.17 to 0.18 wide, this set cannot resolve differences of the size measured on studio images (about 0.02).

On the 700 DWSD images from India, all three models fail, with 0.023 to 0.027 mAP@50, precision 0.070–0.075 and recall 0.043–0.047; only PLASTIC (0.089–0.101) and PAPER (0.028–0.035) exceed 0.01 AP@50. The defaults and PSO are both significantly above random search and do not differ from each other. On mAP@50–95, the two significant differences are B−A -0.002 [-0.003, -0.001] and C−B $+0.002$ [$+0.000$, $+0.003$], whose lower bound is 0.0004 before rounding. These differences are a few thousandths on scores near zero, so they carry no practical meaning.

Four causes of the failure are known from the data (Section III-B): no Indian image was used for training, search or model selection; the DWSD images look augmented (rotated, with black corners and color tints), unlike any training image;

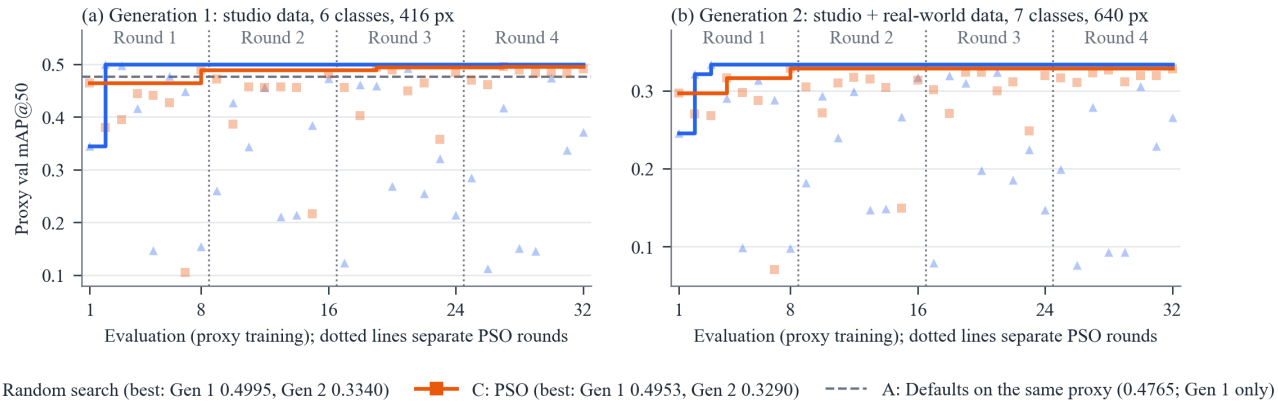


Fig. 3. Search convergence on the proxy fitness (validation mAP@50 after 12 epochs on 40% of the training split). Pale markers are individual evaluations (triangles: random search; squares: PSO); solid lines are the best score so far. Dotted lines separate the four PSO rounds of eight particles; random search has no rounds and is plotted in sampling order. The dashed line in (a) is the score of the defaults on the same proxy; Generation 2 has no such run. Proxy scores are not comparable between the panels, because the validation sets, classes and image sizes differ.

TABLE VI
GENERATION 1 TEST RESULTS ON THE STUDIO TEST SPLIT (1,046 IMAGES)

Arm	Val mAP@50	Test mAP@50 [95% CI]	Test mAP@50–95 [95% CI]	P	R	ms/img
A: Defaults	0.657	0.676 [0.639, 0.722]	0.474 [0.441, 0.518]	0.747	0.581	31.4
B: Random search	0.643	0.672 [0.634, 0.717]	0.475 [0.441, 0.519]	0.746	0.587	38.3
C: PSO	0.657	0.668 [0.627, 0.717]	0.471 [0.437, 0.516]	0.747	0.586	31.7

Pair	ΔmAP@50 [95% CI]	$P(\Delta > 0)$	ΔmAP@50–95 [95% CI]	$P(\Delta > 0)$	Verdict
B−A	−0.004 [−0.016, +0.009]	0.30	+0.001 [−0.008, +0.010]	0.59	not significant
C−A	−0.008 [−0.021, +0.007]	0.16	−0.003 [−0.012, +0.007]	0.34	not significant
C−B	−0.004 [−0.015, +0.007]	0.24	−0.003 [−0.011, +0.004]	0.19	not significant

Val: best checkpoint on the validation split after 100 epochs. CIs: 1,000 paired bootstrap resamples of the test images. P and R at the confidence of maximum F1. ms/img: median batch-1 latency at 416 px on a laptop CPU (Intel Core Ultra 5 125H). $P(\Delta > 0)$: share of resamples in which the first arm of the pair scored higher. A difference is significant only if its CI excludes 0.

TABLE VII
GENERATION 2 TEST RESULTS PER TEST SET: MAP@50 AND MAP@50–95 [95% CI] PER ARM, AND PAIRWISE DIFFERENCES IN MAP@50

	All 1,514 img., 8,192 boxes	Studio 1,046 img., 7,413 boxes	Real-world 468 img., 779 boxes	India (DWSD) 700 img., 4,502 boxes
Test mAP@50 [95% CI]				
A: Defaults	0.479 [0.453, 0.513]	0.571 [0.529, 0.619]	0.310 [0.278, 0.440]	0.027 [0.024, 0.031]
B: Random search	0.497 [0.471, 0.531]	0.595 [0.557, 0.639]	0.298 [0.271, 0.407]	0.023 [0.021, 0.027]
C: PSO	0.500 [0.474, 0.534]	0.590 [0.550, 0.635]	0.310 [0.279, 0.415]	0.027 [0.024, 0.031]
ΔmAP@50 [95% CI] (bold: significant, CI excludes 0)				
B−A	**+0.017** [+0.006, +0.030]	**+0.024** [+0.009, +0.038]	−0.012 [−0.118, +0.053]	**−0.003** [−0.006, −0.001]
C−A	**+0.020** [+0.011, +0.032]	**+0.019** [+0.005, +0.033]	+0.001 [−0.111, +0.065]	+0.000 [−0.003, +0.003]
C−B	+0.003 [−0.008, +0.015]	−0.005 [−0.018, +0.009]	+0.013 [−0.015, +0.036]	**+0.003** [+0.001, +0.006]
Test mAP@50–95 [95% CI]				
A: Defaults	0.329 [0.309, 0.358]	0.386 [0.356, 0.425]	0.254 [0.224, 0.371]	0.015 [0.013, 0.018]
B: Random search	0.342 [0.322, 0.372]	0.404 [0.374, 0.442]	0.238 [0.213, 0.330]	0.013 [0.012, 0.015]
C: PSO	0.343 [0.323, 0.374]	0.398 [0.367, 0.439]	0.248 [0.220, 0.336]	0.015 [0.013, 0.017]

All test sets were scored once, at 640 px, with 1,000 paired bootstrap resamples per test set. "All" is the whole Generation 2 test split (studio plus real-world); India is DWSD, used for testing only. The mAP@50–95 differences give the same verdict as the mAP@50 differences in every cell. Validation mAP@50: A 0.454, B 0.473, C 0.466.

DWSD's boxes come from mask regions, so touching items of one class merge into one large box that per-item detections overlap poorly, and even correct detections often count as misses; and the mapping from mask values to classes was inferred from the images, with six of the fifteen values uncertain. The India score thus measures the difference in labeling style as well as the models. Fig. 6 shows the three models on two studio, two real-world and two DWSD test images. On the two real-world images, which hold eight objects each, every model finds at most three objects of the first and none of the second. On the DWSD images each model predicts at most one box, against 4 and 19 ground-truth boxes, and on the first of them all three models box the region that the ground truth labels PAPER but call it PLASTIC.

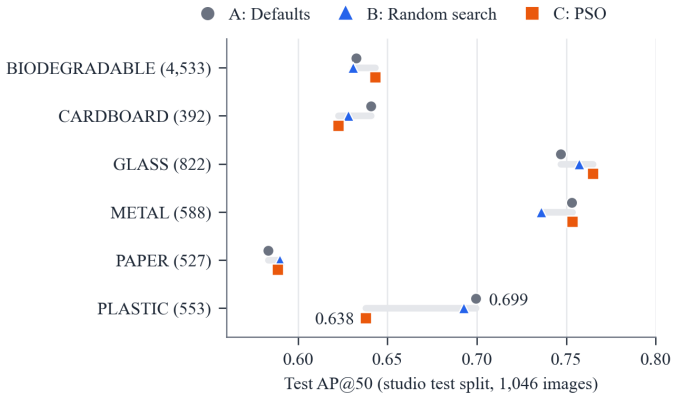


Fig. 4. Per-class test AP@50 of the three Generation 1 models on the studio test split. Numbers in parentheses are the test boxes per class as counted by the evaluator (7,415 in total, including 2 zero-size BIODEGRADABLE boxes); the gray bar spans the three arms' values. PSO is lowest on PLASTIC; elsewhere the arms are within 0.02 of each other.

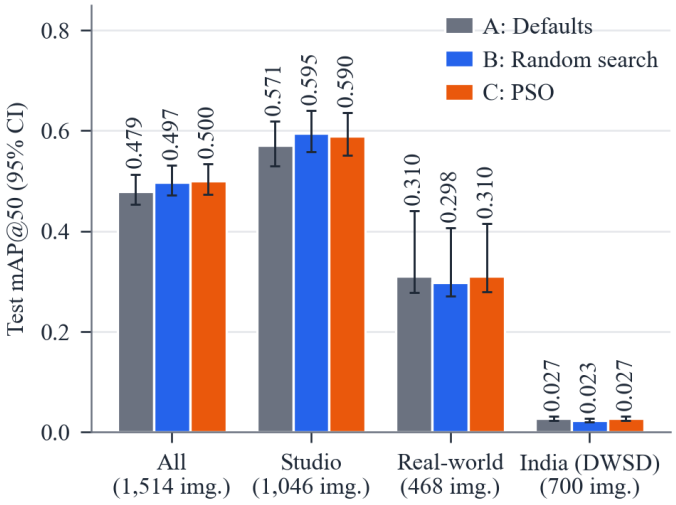


Fig. 5. Test mAP@50 of the three Generation 2 models per test set, with 95% paired-bootstrap confidence intervals. "All" combines the studio and real-world test images; India (DWSD) was used for testing only. The real-world intervals are lopsided because that set holds a single BIODEGRADABLE box.

The two generations differ in classes, image size and training data, so Tables VI and VII are not one benchmark. Table VIII therefore scores all six final models on the same studio and real-world test images, restricted to the six shared classes (Section III-I). On the real-world images, every Generation 2 arm is significantly better than every Generation 1 arm: 0.338–0.353 mAP@50 against 0.038–0.096, about four to nine times as high. On the studio images, the order reverses: every Generation 1 arm is significantly better than every Generation 2 arm, by 0.074 to 0.105. Within each generation, no real-world difference is significant (e.g., Generation 1 C−A +0.048 [−0.020, +0.163], Generation 2 C−B +0.013 [−0.019, +0.042]), and the studio verdicts match Tables VI and VII. The 0.096 of Generation 1 C and its wide interval come largely from the single real-world BIODEGRADABLE box, on which it scores 0.332 AP@50. Generation 2 scores higher on real-world images here than in Table VII (A 0.353 against 0.310) only because OTHER is left out of the mean. As a check, Generation 1 reproduces its own studio test result within 0.0004 mAP@50.

TABLE VIII
GENERATION 1 VERSUS GENERATION 2 ON THE SAME TEST IMAGES: TEST MAP@50 [95% CI] ON THE SIX SHARED CLASSES

Model (px)	Real-world (468 img.)	Studio (1,046 img.)
G1 A (416)	0.047 [0.038, 0.080]	0.676 [0.641, 0.719]
G1 B (416)	0.038 [0.031, 0.069]	0.672 [0.636, 0.716]
G1 C (416)	0.096 [0.041, 0.214]	0.669 [0.628, 0.716]
G2 A (640)	0.353 [0.315, 0.504]	0.571 [0.529, 0.619]
G2 B (640)	0.338 [0.304, 0.473]	0.595 [0.557, 0.639]
G2 C (640)	0.352 [0.315, 0.485]	0.590 [0.550, 0.635]
Δ G2−G1	+0.243 to +0.315	−0.105 to −0.074

G1, G2: Generation 1, Generation 2; each model was scored once at its own training image size (px). Real-world: 647 boxes after removing OTHER, with a single BIODEGRADABLE box; studio: 7,413 boxes, no OTHER. CIs: 1,000 paired bootstrap resamples per test set. Δ: range over all nine G2−G1 pairs, every one significant; e.g., real-world G2 B−G1 C +0.243 [+0.134, +0.418], studio G2 A−G1 A −0.105 [−0.125, −0.084]. On the five real-world classes with more than one box, Generation 2 scores 0.239–0.618 AP@50 and Generation 1 0.008–0.117.

Where, then, do the Generation 2 models succeed on real-world photographs? A per-image analysis of the 468 real-world test images counts an image as fully correct when every ground-truth object is matched by a detection of the same class with IoU ≥ 0.5 (greedy, highest confidence first, at a confidence threshold of 0.25) and no detection is left unmatched. Each model is fully correct on 166 (A), 168 (B) and 170 (C) images, 35 to 36% of the set, and all three on the same 111. The successes are almost all single-object photographs: of the 375 images with one object, the models are fully correct on 164 to 167; of the 47 with two objects, on 1 to 4; and of the 46 with three or more, on none. Fig. 7 shows the most favorable cases, chosen by a fixed rule. No image with three or more objects, and only one with two, is fully correct for all three models, so the rule fell back to the two images with at least two objects that at least two models label fully correctly; one of them is a close-up on a plain background, close to studio conditions. The models thus handle an isolated item in a real scene, but not a cluttered scene with several items.

F. Compute Cost

Table IX lists the recorded GPU time of every run on one Tesla T4, covering training only (setup, data copies and idle time are excluded). Including its search, each tuned arm cost about 2.7 to 2.8 times the GPU time of Arm A, which needs no search. The budget was equal in evaluations, not in time: the mean proxy training took 214.3 s for PSO and 238.7 s for random search in Generation 1, and 285.5 s and 259.0 s in Generation 2. The largest Generation 1 item was the proxy-validity check, of which the eight 40-epoch medium runs took 3.84 of 4.36 hours. In Generation 2, about 24 further GPU-hours were lost when two sessions froze and sat idle until the platform's 12-hour limit; they are not in the totals. The one-time test evaluation took 283.1 s on the laptop CPU in

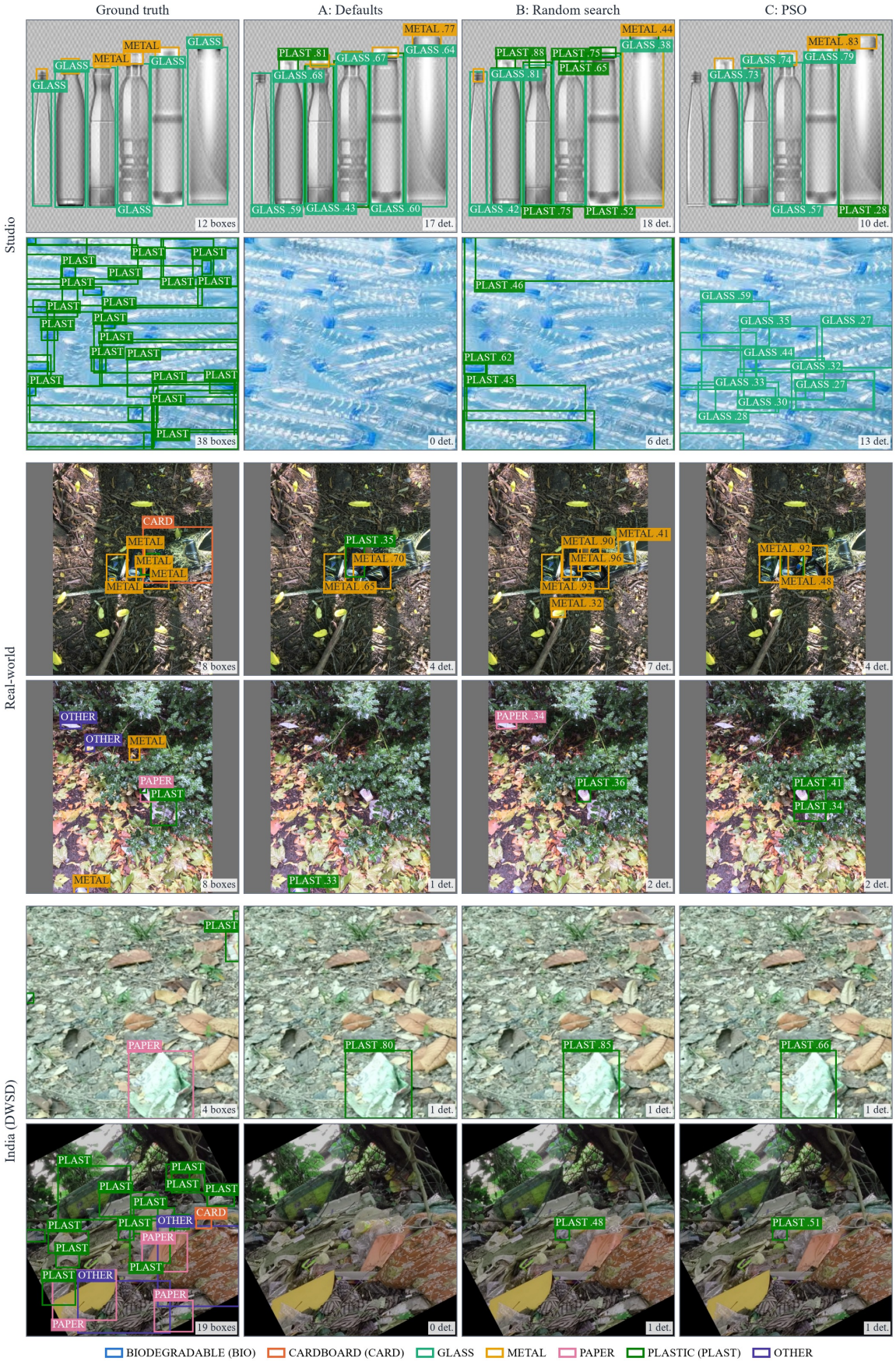


Fig. 6. Detections of the three Generation 2 models on two studio, two real-world and two Indian (DWSD) test images, next to the ground truth. The images were chosen by a fixed rule, not by eye: for each test set, the images with at least three ground-truth boxes were put in a random order with seed 0, and the first two are shown. Every model saw the same images at 640 px, with a confidence threshold of 0.25 and an NMS IoU of 0.7; the real-world rows show the predictions recorded in the per-image analysis of Fig. 7. Each box carries its class and, for predictions, its confidence; a label that would cover another is left out. The corner of each panel gives the number of ground-truth boxes or detections. These panels are illustrations, not evidence; the quantitative results are in Table VII and Fig. 5. The DWSD ground-truth boxes come from mask regions, so touching items of one class can merge into one box.



Fig. 7. Selected success cases on real-world test images: deliberately favorable, not representative. Rows 1–2: the only real-world test images with at least two objects that at least two of the three Generation 2 models label fully correctly (every object found with the right class, no false detection); no image with three or more objects qualifies, and random search (B) misses one object in the first. Rows 3–4: the first two of the 110 single-object images that all three models label fully correctly, in a random order with seed 0. Confidence threshold 0.25, NMS IoU 0.7, 640 px. For scale, each model is fully correct on 166–170 of the 468 real-world test images, on 164–167 of the 375 single-object images and on none of the 46 images with three or more objects.

TABLE IX
RECORDED GPU TIME PER RUN (HOURS, ONE TESLA T4)

Run	Generation 1	Generation 2
Proxy-validity check (16 runs)	4.36	not run
PSO search (32 proxy trainings)	1.91	2.54
Random search (32 proxy trainings)	2.12	2.30
Arm A full training (100 epochs)	1.18	1.56
Arm B full training (100 epochs)	1.15	2.07
Arm C full training (100 epochs)	1.45	1.60
Total recorded	12.18	10.06

Sums of the recorded training wall-clock times. The Generation 2 random search and Arm C training were resumed after a session froze (Arm C after epoch 55); their idle time, about 24 GPU-hours in total, is not included.

Generation 1 and 220.6 s on the T4 for the four Generation 2 test sets.

V. DISCUSSION

A. *Why PSO Did Not Beat Random Search*

In Generation 1, the defaults seem to sit in a broad, flat region of good settings. They beat most proxy evaluations of both searches and trailed the two winners by only 0.019 (PSO) and 0.023 (random search) (Section IV-B); in the proxy check they ranked second of eight at both training lengths, and the four best settings lay within 0.026 after 40 epochs (Table V).

Within that region, the proxy separated poor settings from good ones ($\rho = 0.976$) but could not pick a winner: its Generation 1 winner, random search's setting, gave the lowest validation score after 100 epochs (Table VI). The short proxy may favor fast-learning settings, and the maximum of 32 single-run scores favors scores inflated by chance; we measured neither effect.

The budget was also small: 32 evaluations in six dimensions, eight of them spent by PSO on its random initial swarm. Bergstra and Bengio [10] report that for most data sets only a few hyperparameters really matter and that random search finds models as good as or better than grid search at the same budget. If one or two of our dimensions dominate, 32 independent samples cover them densely, leaving a guided search little to exploit; the below-default initial learning rate of all four search winners fits this view, though per-dimension importance was not measured.

The swarm converged early, raising its mean proxy score above that of random search but not its best (Section IV-B). In Generation 2, its best was the last particle of the random initial swarm (Fig. 3), so PSO in effect offered the best of eight random samples against the best of 32. With three update steps, a global-best swarm concentrates later evaluations around round 1's best point, raising the mean but not the maximum, the only score used.

Why tuning helped only in Generation 2 is less clear. We hypothesize that the new data (real-world photographs, 640 px, a seventh class) moved the good region away from the defaults, so any reasonable search gained about 0.02 mAP@50 on the mixed and studio test sets; with no Generation 2 proxy run of the defaults, this is untested. As each arm was trained once, part of the gain could also be training noise; repeating the proxy check and full trainings for Generation 2 would tell the two apart (Section VI-A).

B. Implications for Tuning Claims in Waste AI

Gap 3 (Section II-E) is that metaheuristic tuning in this area is compared with other metaheuristics or with nothing, using single test numbers; our results show the cost. Against the defaults alone, PSO's significant Generation 2 gain of $+0.020$ mAP@50 would have been credited to the swarm, yet random search gained $+0.017$ at the same budget and the tuned arms cannot be told apart (Table VII): the gain belongs to searching at all. Single numbers would also have misled: the Generation 1 point estimates rank the defaults first and PSO last, and Generation 2 validation ranked random search ahead of PSO, yet neither ordering is significant on test. Conversely, PSO's significant $+0.003$ over random search on India, on scores near 0.02, shows that a difference must also matter in practice.

We therefore recommend a minimum protocol for any claim that a metaheuristic tunes a waste model better: (1) include the defaults and random search at the same number of evaluations, and report time (Section IV-F); (2) check any cheap proxy against longer training before the search, with a threshold fixed in advance; (3) choose each arm's final setting on validation data and score the test split once; (4) report paired confidence intervals on arm differences per test domain, judged against a practically relevant difference; and (5) repeat trainings and searches with several seeds. SwarmSort meets all but the fifth. In equal-budget comparisons such as that of Sayed et al. [9], a random-search arm would cost only one more competitor.

C. Real-World and Indian Deployment

For a smart bin or a municipal service, the data matter far more than the optimizer. Every arm loses about half its studio score on real-world photographs and collapses to 0.023–0.027 mAP@50 on Indian ones (Section IV-E); the largest tuning effect, about 0.02, is less than a tenth of the studio-to-real-world drop (about 0.28) and did not appear on real-world photographs. The strongest evidence is Table VIII, which scores both generations on the same images. Real-world training data raised real-world mAP@50 from 0.038–0.096 (Generation 1) to 0.338–0.353 (Generation 2), every Generation 2 arm significantly above every Generation 1 arm, whereas tuning made no significant real-world difference in either generation. The price was a significant studio loss of 0.074 to 0.105. Its three likely causes, which our runs do not separate, are that Generation 2 kept only 41% of the Generation 1 studio training images, that the same small YOLOv8n shares its capacity with real-world scenes and a seventh class, and that Generation 2 trained at 640 rather than 416 px and chose its checkpoint on mixed validation data. Local training data will therefore help more than a better search, which is why Indian data lead the roadmap (Section VI-B). Clutter is the second obstacle: the models label every object correctly on about 44% of the single-object real-world photographs but on none of the 46 with three or more objects (Fig. 7), so a bin that photographs one item at a time is a realistic target today, while sorting a mixed pile is not.

Because DWSD's merged, mask-derived boxes penalize correct per-item detections (Section IV-E), a test set must be labeled like the training data before its score can be read as accuracy; relabeling part of DWSD item by item would separate labeling style from model error.

Indian rules also constrain the classes: Rule 15(h) of the Solid Waste Management Rules, 2016 [1] requires green, white and black bins for bio-degradable, recyclable and other waste. BIODEGRADABLE maps to green and the material classes, insofar as their items are recyclable, to white, but black corresponds to OTHER, every arm's weakest class (AP@50 0.050 to 0.063), where a bin built on these models would be least reliable.

D. Limitations and Threats to Validity

Each arm was trained once (seed 42) and each search run once (PSO seed 42, random-search seed 123), so the intervals cover test sampling only, and another seed could change the small arm differences. No p-values or multiple-comparison corrections were computed for the twelve pairwise comparisons per metric in Generation 2 (three pairs, four test sets), per-class AP has no interval, and the real-world test set's single BIODEGRADABLE box makes its intervals wide and lopsided.

The proxy check used eight settings, Generation 1 only and a 40-epoch rather than a 100-epoch reference, so proxy validity in Generation 2 is assumed. The proxy scored last-epoch weights, whereas the final models used the best validation checkpoint (`best.pt`). The Generation 2 Arm C training, resumed after epoch 55 when a session froze, kept the same

weights and optimizer state but is not bit-identical to an uninterrupted run.

DWSD's class mapping was inferred from the masks, with six of the fifteen mask values uncertain, and its boxes are connected regions rather than single items. Only six training hyperparameters were searched, and the budget was equal in evaluations, not in time. No external baseline, such as the pipeline of Majchrowska et al. [6], was run on our data, so our scores cannot be ranked against prior work. Finally, the literature review was targeted rather than systematic (Section II-A), so the gaps (Section II-E) hold only for the literature we reviewed.

VI. Future Work: Possible Improvements

Each improvement answers a limitation (Section V-D) or a measured result; Table X orders them by expected value for effort.

A. The Optimization Study

Three training seeds per arm would let the intervals cover training noise, and several search seeds per method would show whether PSO's result repeats. Because the small proxy gaps among the top settings vanished after full training, multi-fidelity search such as successive halving or Hyperband [38] should promote the best candidates to longer runs before a winner is chosen. Because the swarm converged early (in Generation 2 it never improved after round 1), PSO variants could keep it exploring: the constriction factor of Clerc and Kennedy [37], a ring topology, adaptive inertia, or more particles and rounds.

Other optimizers should join as further arms against the same baselines at 32 evaluations: a genetic or evolutionary algorithm [45], differential evolution [46] and Bayesian optimization such as Optuna's tree-structured Parzen estimator [47]. This also answers why PSO and not, say, the beluga whale optimizer of Sayed et al. [9]: the protocol is optimizer-agnostic, and its value is that every candidate meets the same baselines. A wider search space (color augmentation, translation, loss gains, image size) would test whether six settings left too little room to beat the defaults, and a domain-aware fitness scored on real-world validation images targets tuning gains that did not reach real-world photographs. Finally, the proxy check should be repeated for Generation 2 against a 100-epoch reference, with a defaults run testing the hypothesis of Section V-A, and the statistics strengthened with Holm's correction, per-class AP intervals and stored bootstrap resamples.

B. Data and the Indian Domain

The main next step is Indian training data with per-item boxes (part of DWSD or a new collection, with a held-out test part), which addresses the DWSD failure and the labeling mismatch behind part of it; since Generation 2's real-world data cost studio accuracy (Section V-C), it should be added by domain-aware training, for example keeping all studio images or using a larger model. The weak OTHER class needs more examples, a split into clearer classes, or a merge into existing ones. Clutter augmentations such as copy-paste [48], smaller object scales, blur and lighting changes target the small, overlapping and partly hidden items of real-world photographs. Finally, the classes could follow the three streams that the SWM Rules 2016 require, bio-degradable, non-bio-degradable and domestic hazardous [1], so that each detection maps to a bin.

C. The Model

Larger or newer detectors (YOLOv8s, YOLOv8m, YOLO11 [49]) with the same settings would show how much of the real-world gap is due to model capacity, at a cost in speed on edge devices. Sliced inference [50] on overlapping tiles suits wide photographs with many small items, and confidence calibration such as temperature scaling [51] would make a score of 0.5 mean roughly 50% correct, which matters when a bin acts on a threshold.

D. Combining the Three Models

On point estimates, no arm leads on every Generation 2 test set (Table VII). On real-world photographs, where the three mAP@50 values lie within 0.012, the arms differ more in their parts: precision at the maximum-F1 confidence ranges from 0.366 (B) to 0.556 (C), PAPER AP@50 from 0.239 (A) to 0.330 (C), and random search leads on GLASS (0.296). These values lack intervals and model agreement was not measured, but they suggest partly different mistakes, so combining the arms, with no extra training, could beat each one alone.

Three combinations should be compared: box fusion, which pools the boxes of A, B and C and merges overlapping ones by weighted boxes fusion [52] or non-maximum suppression; majority voting, which keeps a box only if at least two models find it (IoU ≥ 0.5) to raise precision and can flag single-model boxes as uncertain instead of discarding them; and weighted voting by each model's validation mAP@50, never by test scores, which would leak the test split. As a soft-computing extension, a search yields a population of good settings (the final swarm and the best random samples), so training and combining the top-k uses the swarm as an ensemble generator; since the converged swarm may lack diversity, top random samples should be mixed in. Each ensemble should be compared with the best single arm, chosen on validation, by the same paired bootstrap on every test set. Its cost is inference time: about three times as long, roughly 100 ms per image on the laptop CPU against 31 to 38 ms for one Generation 1 model.

E. Deployment

Per-class confidence thresholds chosen on validation data could replace the single threshold, since real-world precision is low and AP varies widely by class. Because our latencies come only from a laptop CPU and a cloud GPU, the models should be exported to ONNX or TFLite and timed on a Raspberry Pi, the class of hardware Alshammari et al. [15] use for a smart-bin classifier, and, since real sorting is continuous, tested on live video.

TABLE X
FUTURE-WORK ROADMAP, ORDERED BY EXPECTED VALUE FOR EFFORT

Priority	Improvement	Addresses	Effort
1	Indian training data with per-item boxes	DWSD failure (0.023–0.027 mAP@50)	Large
2	Ensemble of A, B and C (fusion or voting)	Arms differ per class and test set; no arm wins everywhere	Small (no training)
3	Three training seeds per arm, several search seeds	Tuning effect vs. training noise	Medium
4	Domain-aware fitness (real-world validation)	Tuning gains did not carry to real-world photos	Medium
5	Multi-fidelity search (Hyperband)	Proxy gaps vanish at 100 epochs	Medium
6	GA, DE and Bayesian optimization at the same budget	"Why PSO?"	Medium
7	Fix the OTHER class	AP@50 0.050–0.063	Medium
8	Larger detector; sliced inference	Small items, look-alike materials	Medium
9	Edge export and latency on a Raspberry Pi	Real smart-bin hardware	Medium
10	Swarm as an ensemble generator (top-k settings)	Uses the whole PSO population	Medium

Effort: Small = hours, no GPU training; Medium = about a day with some GPU time; Large = several days or a new data pipeline. GA = genetic algorithm; DE = differential evolution.

VII. CONCLUSION

SwarmSort compared a particle swarm, random search and the defaults for tuning a lightweight waste detector at an equal budget of 32 proxy trainings.

RQ1 asked whether PSO beats the other two on a held-out test set. It never meaningfully beat random search. In Generation 1 (studio data), the arms were statistically tied (0.676, 0.672 and 0.668 mAP@50). In Generation 2, both tuned arms beat the defaults on the mixed test set (random search by $+0.017$, PSO by $+0.020$), but PSO and random search stayed indistinguishable on the mixed, studio and real-world test sets; their only significant difference, $+0.003$ on India, lies on scores near 0.02 and is practically meaningless. RQ2 asked whether the short proxy ranks settings like longer training. It did ($\rho = 0.976$, $n = 8$, Generation 1 only), but its small gaps between the best settings vanished after full training. RQ3 asked whether the answer changes with the domain. It does: tuning gains did not reach real-world photographs (0.298 to 0.310), and all arms failed on the Indian DWSD photographs (0.023 to 0.027), partly because of DWSD's merged boxes. Real-world training data, by contrast, raised real-world mAP@50 from 0.038–0.096 (Generation 1) to 0.338–0.353 (Generation 2) on the same images, at a studio cost of 0.074 to 0.105.

Two messages follow. A claim that a metaheuristic tunes a model better needs the defaults and random search at an equal budget and confidence intervals on a test split used once; without random search, our Generation 2 gain for PSO would have been credited to the swarm. And for waste detection in real, especially Indian, scenes, training data matter far more than the optimizer. The next steps (Section VI) are led by Indian training data with per-item boxes and an ensemble of the three models.

DATA AND CODE AVAILABILITY

All data used in this study are public. Generation 1 uses the Kaggle dataset "Garbage Detection – 6 Waste Categories" [40] (CC BY 4.0). Generation 2 adds TACO [17] (CC BY 4.0) and the Humans in the Loop Recycling dataset [41] (CC0). The Indian test set was built from DWSD [13] (CC BY 4.0), which is available from Mendeley Data at https://doi.org/10.17632/gr99ny6b8p.1. The code, the split file, all search logs and the result files are available in the project repository, https://github.com/CrossAnushka/Particle-Swarm-Optimization-for-Waste-Detection, together with the dataset-build configuration that maps every source to the shared classes. The repository also contains the trained weights of all six models, the three arms of each of the two generations, in `app/models/` (Generation 1) and `app/models/gen2/` (Generation 2).

REFERENCES

[1] Ministry of Environment, Forest and Climate Change, Government of India, "Solid waste management rules, 2016," Notification S.O. 1357(E), Gazette of India, Extraordinary, Part II, Section 3, Sub-section (ii), No. 861, Apr. 2016. [Online]. Available: https://cpcb.gov.in/uploads/MSW/SWM_2016.pdf

[2] M. Abdallah, M. Abu Talib, S. Feroz, Q. Nasir, H. Abdalla, and B. Mahfood, "Artificial intelligence applications in solid waste management: A systematic research review," *Waste Management*, vol. 109, pp. 231–246, 2020.

[3] B. Fang, J. Yu, Z. Chen, A. I. Osman, M. Farghali, I. Ihara, E. H. Hamza, D. W. Rooney, and P.-S. Yap, "Artificial intelligence for waste management in smart cities: a review," *Environmental Chemistry Letters*, vol. 21, no. 4, pp. 1959–1989, 2023.

[4] M. Yang and G. Thung, "Classification of trash for recyclability status," Stanford University, CS229 Project Report, 2016. [Online]. Available: https://cs229.stanford.edu/proj2016/report/ThungYang-ClassificationOfTrashForRecyclabilityStatus-report.pdf

[5] V. Kaya, "Classification of waste materials with a smart garbage system for sustainable development: a novel model," *Frontiers in Environmental Science*, vol. 11, p. 1228732, 2023.

[6] S. Majchrowska, A. Mikołajczyk, M. Ferlin, Z. Klawikowska, M. A. Plantykow, A. Kwasigroch, and K. Majek, "Deep learning-based waste detection in natural and urban environments," *Waste Management*, vol. 138, pp. 274–284, 2022.

[7] J. Kennedy and R. Eberhart, "Particle swarm optimization," in *Proceedings of ICNN'95 - International Conference on Neural Networks*, vol. 4. Perth, WA, Australia: IEEE, 1995, pp. 1942–1948.

[8] K. Aguerchi, Y. Jabrane, M. Habba, and A. Hajjam El Hassani, "A CNN hyperparameters optimization based on particle swarm optimization for mammography breast cancer classification," *Journal of Imaging*, vol. 10, no. 2, p. 30, 2024.

[9] G. I. Sayed, M. Abd Elfattah, A. Darwish, and A. E. Hassanien, "Intelligent and sustainable waste classification model based on multi-objective beluga whale optimization and deep learning," *Environmental Science and Pollution Research*, vol. 31, no. 21, pp. 31 492–31 510, 2024.

[10] J. Bergstra and Y. Bengio, "Random search for hyper-parameter optimization," *Journal of Machine Learning Research*, vol. 13, no. 10, pp. 281–305, 2012. [Online]. Available: https://jmlr.org/papers/v13/bergstra12a.html

[11] W. Lu and J. Chen, "Computer vision for solid waste sorting: A critical review of academic research," *Waste Management*, vol. 142, pp. 29–43, 2022.

[12] M. Córdova, A. Pinto, C. C. Hellevik, S. A.-A. Alaliyat, I. A. Hameed, H. Pedrini, and R. d. S. Torres, "Litter detection with deep learning: A comparative study," *Sensors*, vol. 22, no. 2, p. 548, 2022.

[13] A. Ali, S. Acharjee, M. M. Sk, S. Z. Alharthi, S. S. Chaudhuri, and A. Akhunzada, "DWSD: Dense waste segmentation dataset," *Data in Brief*, vol. 59, p. 111340, 2025.

[14] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," Software, version 8.0.0, AGPL-3.0 licence, 2023. [Online]. Available: https://github.com/ultralytics/ultralytics

[15] A. Alshammari, S. Alkhabbaz, R. Almutairi, H. Alhabad, S. Alhamdan, and I. Boulkaibet, "Smart waste classification system using deep learning for automated recycling," in *2025 6th International Conference on Bio-engineering for Smart Technologies (BioSMART)*. Paris, France: IEEE, 2025, pp. 1–6.

[16] N. V. Kumsetty, A. B. Nekkare, S. Kamath S., and A. Kumar M., "TrashBox: Trash detection and classification using quantum transfer learning," in *2022 31st Conference of Open Innovations Association (FRUCT)*, 2022, pp. 125–130.

[17] P. F. Proença and P. Simões, "TACO: Trash annotations in context for litter detection," arXiv preprint arXiv:2003.06975, 2020.

[18] D. Bashkirova, M. Abdelfattah, Z. Zhu, J. Akl, F. Alladkani, P. Hu, V. Ablavsky, B. Calli, S. A. Bargal, and K. Saenko, "ZeroWaste dataset: Towards deformable object segmentation in cluttered scenes," in *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 21 115–21 125.

[19] D. Yudin, N. Zakharenko, A. Smetanin, R. Filonov, M. Kichik, V. Kuznetsov, D. Larichev, E. Gudov, S. Budennyy, and A. Panov, "Hierarchical waste detection with weakly supervised segmentation in images from recycling plants," *Engineering Applications of Artificial Intelligence*, vol. 128, p. 107542, 2024.

[20] M. Kraft, M. Piechocki, B. Ptak, and K. Walas, "Autonomous, onboard vision-based trash and litter detection in low altitude aerial images collected by an unmanned aerial vehicle," *Remote Sensing*, vol. 13, no. 5, p. 965, 2021.

[21] J. Hong, M. Fulton, and J. Sattar, "TrashCan: A semantically-segmented dataset towards visual detection of marine debris," arXiv preprint arXiv:2007.08097, 2020.

[22] Y. Cheng, J. Zhu, M. Jiang, J. Fu, C. Pang, P. Wang, K. Sankaran, O. Onabola, Y. Liu, D. Liu, and Y. Bengio, "FloW: A dataset and benchmark for floating waste detection in inland waters," in *2021 IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021, pp. 10 933–10 942.

[23] T. Wang, Y. Cai, L. Liang, and D. Ye, "A multi-level approach to waste object segmentation," *Sensors*, vol. 20, no. 14, p. 3816, 2020.

[24] M. Fulton, J. Hong, M. J. Islam, and J. Sattar, "Robotic detection of marine litter using deep visual detection models," in *2019 International Conference on Robotics and Automation (ICRA)*, 2019, pp. 5752–5758.

[25] D. O. Melinte, A.-M. Travediu, and D. N. Dumitriu, "Deep convolutional neural networks object detector for real-time waste identification," *Applied Sciences*, vol. 10, no. 20, p. 7301, 2020.

[26] Y.-H. Liao and J.-G. Juang, "Real-time UAV trash monitoring system," *Applied Sciences*, vol. 12, no. 4, p. 1838, 2022.

[27] M. Bartolo, K. Makantasis, and D. Seychell, "Learning using privileged information for litter detection," arXiv preprint arXiv:2508.04124, 2025.

[28] A. Sánchez-Ferrer, J. J. Valero-Mas, A. J. Gallego, and J. Calvo-Zaragoza, "An experimental study on marine debris location and recognition using object detection," *Pattern Recognition Letters*, vol. 168, pp. 154–161, 2023.

[29] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. Las Vegas, NV, USA: IEEE, 2016, pp. 779–788.

[30] S. Kumar, D. Yadav, H. Gupta, O. P. Verma, I. A. Ansari, and C. W. Ahn, "A novel YOLOv3 algorithm-based deep learning approach for waste segregation: Towards smart waste management," *Electronics*, vol. 10, no. 1, p. 14, 2021.

[31] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, "Microsoft COCO: Common objects in context," in *Computer Vision – ECCV 2014*, ser. Lecture Notes in Computer Science. Cham: Springer International Publishing, 2014, pp. 740–755.

[32] G. Mittal, K. B. Yagnik, M. Garg, and N. C. Krishnan, "SpotGarbage: Smartphone app to detect garbage using deep learning," in *Proceedings of the 2016 ACM International Joint Conference on Pervasive and Ubiquitous Computing (UbiComp '16)*, 2016, pp. 940–945.

[33] H. Panwar, P. K. Gupta, M. K. Siddiqui, R. Morales-Menendez, P. Bhardwaj, S. Sharma, and I. H. Sarker, "AquaVision: Automating the detection of waste in water bodies using deep transfer learning," *Case Studies in Chemical and Environmental Engineering*, vol. 2, p. 100026, 2020.

[34] M. R. Isalm, N. B. Mahabub, M. J. A. Rafi, P. K. Roy, T. Roy, M. T. Islam, and M. A. Razzak, "UGV-NBWASTE: An oriented dataset for non-biodegradable waste in Bangladesh," *Data in Brief*, vol. 60, p. 111559, 2025.

[35] S. R. Mandhati, N. L. Deshapriya, C. L. Mendis, K. Gunasekara, F. Yrle, A. Chaksan, and S. Sanjeev, "pLitterStreet: Street level plastic litter detection and mapping," arXiv preprint arXiv:2401.14719, 2024.

[36] Y. Shi and R. Eberhart, "A modified particle swarm optimizer," in *1998 IEEE International Conference on Evolutionary Computation Proceedings. IEEE World Congress on Computational Intelligence*. Anchorage, AK, USA: IEEE, 1998, pp. 69–73.

[37] M. Clerc and J. Kennedy, "The particle swarm - explosion, stability, and convergence in a multidimensional complex space," *IEEE Transactions on Evolutionary Computation*, vol. 6, no. 1, pp. 58–73, 2002.

[38] L. Li, K. Jamieson, G. DeSalvo, A. Rostamizadeh, and A. Talwalkar, "Hyperband: A novel bandit-based approach to hyperparameter optimization," *Journal of Machine Learning Research*, vol. 18, no. 185, pp. 1–52, 2018. [Online]. Available: https://jmlr.org/papers/v18/16-558.html

[39] W.-L. Mao, W.-C. Chen, C.-T. Wang, and Y.-H. Lin, "Recycling waste classification using optimized convolutional neural network," *Resources, Conservation and Recycling*, vol. 164, p. 105132, 2021.

[40] viswaprakash1990, "Garbage detection – 6 waste categories," Kaggle dataset, CC BY 4.0, accessed: Sep. 30, 2026. [Online]. Available: https://www.kaggle.com/datasets/viswaprakash1990/garbage-detection

[41] Humans in the Loop, "Recycling dataset," Kaggle dataset, CC0, accessed: Oct. 7, 2026. [Online]. Available: https://www.kaggle.com/datasets/humansintheloop/recycling-dataset

[42] M. D. McKay, R. J. Beckman, and W. J. Conover, "A comparison of three methods for selecting values of input variables in the analysis of output from a computer code," *Technometrics*, vol. 21, no. 2, pp. 239–245, 1979.

[43] C. Spearman, "The proof and measurement of association between two things," *The American Journal of Psychology*, vol. 15, no. 1, pp. 72–101, 1904.

[44] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. Springer US, 1993.

[45] S. R. Young, D. C. Rose, T. P. Karnowski, S.-H. Lim, and R. M. Patton, "Optimizing deep learning hyper-parameters through an evolutionary algorithm," in *Proceedings of the Workshop on Machine Learning in High-Performance Computing Environments (MLHPC)*, 2015, pp. 1–5.

[46] R. Storn and K. Price, "Differential evolution – a simple and efficient heuristic for global optimization over continuous spaces," *Journal of Global Optimization*, vol. 11, pp. 341–359, 1997.

[47] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2019, pp. 2623–2631.

[48] G. Ghiasi, Y. Cui, A. Srinivas, R. Qian, T.-Y. Lin, E. D. Cubuk, Q. V. Le, and B. Zoph, "Simple copy-paste is a strong data augmentation method for instance segmentation," in *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021, pp. 2917–2927.

[49] G. Jocher and J. Qiu, "Ultralytics YOLO11," Software, version 11.0.0, AGPL-3.0 licence, 2024. [Online]. Available: https://github.com/ultralytics/ultralytics

[50] F. C. Akyon, S. O. Altinuc, and A. Temizel, "Slicing aided hyper inference and fine-tuning for small object detection," in *2022 IEEE International Conference on Image Processing (ICIP)*. Bordeaux, France: IEEE, 2022, pp. 966–970.

[51] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proceedings of the 34th International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 70, 2017, pp. 1321–1330. [Online]. Available: https://proceedings.mlr.press/v70/guo17a.html

[52] R. Solovyev, W. Wang, and T. Gabruseva, "Weighted boxes fusion: Ensembling boxes from different object detection models," *Image and Vision Computing*, vol. 107, p. 104117, 2021.